

ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 18/05

- Funciones: Ventajas
- Diseño Modular
- Tipos de Subprogramas.
- Funciones.
- Procedimientos.
- Parametros: De entrada y De salida y de Entrada/Salida
- Funciones recursivas

2

Contraseña del estudiante

wy9ex7

3

Los beneficios de usar Funciones

- **Permitir la programación modular:** divide el problema en partes más simples
- **Hacer el programa más legible:** permite entender mejor el código al agrupar varias sentencias bajo un nombre de función, el código queda más organizado, posibilita encontrar más rápido los errores
- **Reuso de código:** permite ganar tiempo al no tener que escribir las funciones reiteradamente, evita la repetición de código y reduce el tamaño del programa
- **Trabajar con librerías:** permite hacer uso de las funciones que ya están codificadas, reduce el tiempo de codificación
- **Mejor uso de la memoria:** al no tener que reservar espacio para una misma función reiteradas veces en un programa

4

Diseño Modular

- Construir un programa mediante pequeñas piezas o componentes
 - Esas piezas más pequeñas son llamadas **módulos**
- Cada pieza es más manejable que el programa original.

5

Estructura de una función en C++

- Una función es un conjunto de sentencias que se puede llamar desde cualquier parte del programa.
 - Es un fragmento de programa parametrizado, que efectúa unos cálculos y:
 - Devuelve un valor como resultado o,
 - Tiene efectos laterales (modificación de variables globales o argumentos, volcado de información en pantalla, etc.) o,
 - Ambas cosas.
- Para trabajar con funciones se requiere:
 - Prototipo
 - Definición
 - Llamada (activación, ejecución).

6

Prototipo de Funciones

Se usa para validar funciones

- Nombre de la función
- Parámetros que la función recibe
- Tipo de retorno: tipo de dato que la función retorna
- Termina con un punto y coma

El prototipo sólo se necesita si la definición de la función viene después de su uso en el programa

Ejemplo: Prototipo de una función max:

- **int max(int x, int y, int z);**
- Recibe 3 enteros
- Retorna un entero

7

Prototipo de Funciones

```
#include <stdio.h>
```

```
int cuadrado (int x);
int sumar (int a, int b);
float entrada (void);
```

```
int main()
{
    .....
    return 0;
}
```

Prototipos de funciones

8

Estructura de Funciones

tipo-de-retorno nombre-de-funcion (lista-parametros)

{

declaraciones y sentencias;

}

- **Nombre de función:** cualquier identificador disponible
- **Tipo de retorno:** tipo de dato devuelto como resultado
 - void – indica que la función no retorna nada (es un procedimiento)
- **Lista de parámetros:** parámetros declarados, separados por coma.
 - Para cada parámetro, debe indicarse un tipo explícitamente y un nombre.
 - Para indicar que una función no tiene argumentos se usa paréntesis vacíos ().

9

Llamada de Funciones

Las funciones para ejecutarse necesitan ser llamadas o invocadas.

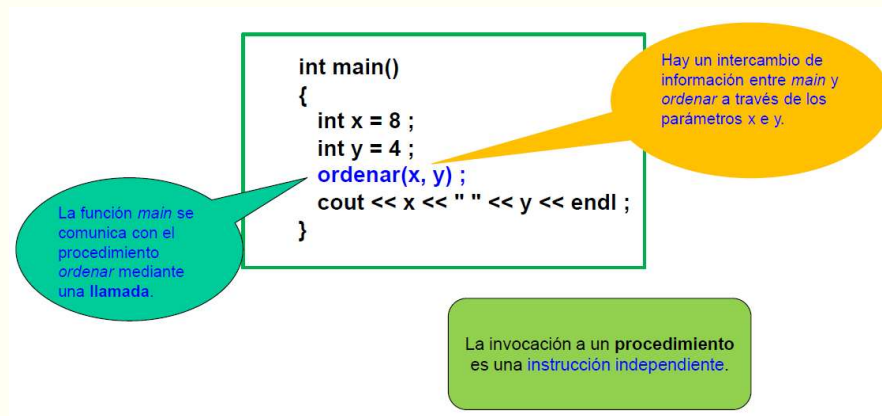
Cuando una función es llamada:

- Se ejecuta desde el principio, y
- Cuando termina retorna el control a la función que la llamó.

10

Tipo de Subprogramas: Procedimientos

En este ejemplo se invoca al procedimiento ordenar para conseguir que, como resultado, el menor de dos números quede almacenado en una variable x y el mayor en una variable y.



11

Procedimientos y Funciones

- Tanto los procedimientos como las funciones se utilizan para realizar un cálculo y simplifican el diseño del programa principal.
- En ambos casos hay una invocación al subprograma correspondiente y un intercambio de información entre el que llama y el subprograma llamado.
- La única diferencia entre ambos tipos de subprogramas está en la forma de hacer las llamadas.

12

Procedimientos y Funciones

- En C++ tanto las funciones como los procedimientos se definen todos como funciones, y se distinguirán según tengan retorno o no:
 - Funciones: tienen un tipo de retorno
 - Procedimientos: tienen retorno void

13

Procedimientos y Funciones

- El intercambio de información en la llamada a subprogramas no difiere del intercambio de información en una comunicación general entre dos entidades. La información puede fluir en uno u otro sentido, o en ambos. Dicho intercambio de información se realiza a través de los parámetros.
- Dependiendo del contexto en que se utilicen, se habla de parámetros formales (los que aparecen en la definición del subprograma), o de parámetros actuales (o reales, los que aparecen en la llamada al subprograma)
- El intercambio de información se analiza desde el punto de vista del subprograma llamado. Se tienen parámetros:
 - de entrada
 - de salida
 - de entrada/salida.

14

Parámetros de entrada

- Son aquellos que se utilizan para recibir la información necesaria para realizar un proceso.
- **Los parámetros de entrada se definen mediante paso por valor.** Ello significa que los parámetros formales son variables independientes, que toman sus valores iniciales como copias de los valores de los parámetros actuales de la llamada en el momento de la invocación al subprograma.
- El parámetro actual puede ser cualquier expresión cuyo tipo sea compatible con el tipo del parámetro formal correspondiente.
- Se declaran especificando el tipo de dato y el identificador asociado.

15

Parámetros de entrada

Paso de parámetros a funciones por VALOR (o por copia)

- Cuando se ejecuta la llamada a la función, ésta recibe una copia de los valores de los parámetros.
- Si se modifica el valor de un parámetro, el cambio sólo afecta a la función y no tiene efecto fuera de ella.
- Se usa cuando la función no necesita modificar el argumento.
- Evita cambios accidentales.

16

Parámetros de entrada - Ejercicio

Paso de parámetros a funciones por VALOR (o por copia)

- Vamos a realizar una función que recibe un monto de dinero, que debe obtener el 2% y retornar (intereses obtenidos en mes).
- Y un programa principal, que pide se ingrese el saldo de una cuenta al inicio del mes, llama a la función, y retorna el saldo final de la cuenta, con el interes ganado en el mes.

17

Parámetros de Salida

- Se utilizan para transferir al programa llamador información producida como parte de la solución realizada por el subprograma.
- Los parámetros de salida se definen mediante paso por referencia.
- Significa que el parámetro formal es una referencia a la variable que se haya especificado como parámetro actual en el momento de la llamada al subprograma.
- Exige que el parámetro actual correspondiente a un parámetro formal por referencia sea una variable.
- Cualquier acción dentro del subprograma que se haga sobre el parámetro formal se realiza sobre la variable referenciada, que aparece como parámetro actual en la llamada al subprograma.
- Se declaran especificando el tipo de dato, el símbolo “ampersand” (&) y el identificador asociado.

18

Parámetros de Salida

```

void dividir (int dividendo, int divisor, int& coc, int& resto);

int main() {
    int cociente, resto;

    dividir(7, 3, cociente, resto);
    cout << "El resultado de dividir 7 entre 3 es: ";
    cout << endl << "Cociente: " << cociente;
    cout << endl << "Resto: " << resto;
    return 0;
}

void dividir (int dividendo, int divisor, int& coc, int& resto){
    coc = dividendo / divisor;
    resto = dividendo % divisor;
}

```

Paso de parámetros por REFERENCIAS

```

El resultado de dividir 7 entre 3 es:
Cociente: 2
Resto: 1
<< El programa ha finalizado: codigo de
<< Presione enter para cerrar esta ventana

```

19

Parámetros de Entrada/Salida

- Son aquellos que se utilizan tanto para recibir información de entrada, necesaria para que el subprograma pueda realizar su computación, como para devolver los resultados obtenidos de la misma.
- Se definen mediante paso por referencia y se declaran igual que los parámetros de salida: especificando el tipo de dato, el símbolo “ampersand” (&) y el identificador asociado.
- Al momento de la llamada se utilizan para ingresar los valores a trabajar y, cuando acaba el subprograma, son utilizados para devolver los resultados.
- Dentro del subprograma los parámetros formales permiten acceder a las variables utilizadas en la llamada.
- Si dentro del subprograma se intercambian los valores de los parámetros, indirectamente se intercambian también los valores de las variables de la llamada.
- Al terminar el subprograma el resultado está almacenado en las variables utilizadas en la llamada.

20

Parámetros de Entrada/Salida

```

void intercambio(int &x, int &y);

int main(){
    int a,b;

    a=3; b=5;

    cout << "a: " << a << " b: " << b << endl;

    intercambio(a,b);

    cout << "a: " << a << " b: " << b << endl;

    return 0;
}

void intercambio( int &x, int &y){
    int aux = x;
    x = y;
    y = aux;
}

```

Parámetros de entrada/salida

```

a: 3 b: 5
a: 5 b: 3

```

Paso de parámetros por REFERENCIAS

Ver diferencia con:
void intercambio (int x, int y)

21

Reglas en el paso de Parámetros

En la llamada a un subprograma se deben cumplir las siguientes normas:

- El número de parámetros actuales debe coincidir con el número de parámetros formales.
- Cada parámetro formal se corresponde con aquel parámetro actual que ocupe la misma posición en la llamada.
- El tipo del parámetro actual debe estar acorde con el tipo del correspondiente parámetro formal.
- Un parámetro formal de salida o entrada/salida (paso por referencia) requiere que el parámetro actual sea una variable.
- Un parámetro formal de entrada (paso por valor) permite que el parámetro actual sea una variable, constante o cualquier expresión.

22

Algoritmos y Estructuras de Datos – AEDD – Asistencia 18/05

Contraseña del estudiante

wy9ex7

23

Funciones Recursivas

La recursividad es una técnica muy potente para la resolución de problemas.

- Es adecuada para problemas que se pueden solucionar usando el mismo algoritmo con versiones más reducidas de los datos.
- En general se obtienen soluciones más elegantes y simples que si se lo soluciona de manera iterativa.
- Existen problemas que por su naturaleza tienen un esquema recursivo o recurrente.

24

Recursividad/Recurrencia

Multiplicación de números naturales	$n * k = n + n * (k-1)$	con $n * 1 = 1$
Factorial de un número	$n! = n * (n-1)!$	con $0! = 1$
Potencia natural de un número	$a^n = a * a^{(n-1)}$	con $a^0 = 1$
Suma de los n primeros naturales	$\text{Suma}(1, n) = n + \text{Suma}(1, n-1)$	con $\text{Suma}(1, 1) = 1$
Cantidad de dígitos de un número	$\text{CantDigitos}(n) = 1 + \text{CantDigitos}(\text{int}(n/10));$ con $\text{CantDigitos}(k) = 1$ si $0 \leq k \leq 9$	
Suma de los dígitos de un número	$\text{SumaDigitos}(n) = (n \% 10) + \text{SumaDigitos}(\text{int}(n/10))$ con $\text{SumaDigitos}(k) = k$ si $0 \leq k \leq 9$	

25

Funciones Recursivas

- Son funciones que se llaman a sí mismas
- Necesitan tener al menos un caso base (se calcula la solución en forma directa, devolviendo un resultado)
- Caso general (no se ha encontrado la solución), luego la función llama a una nueva copia de sí misma (paso de recursión) con un tamaño de problema menor.
- Eventualmente se llega al caso base: esto permite el camino de retorno y la resolución del problema completo.
- El diseño debe garantizar la llegada a uno de los casos base en algún momento.

26

Recursión Vs Iteración

Repetición

- Iteración: utiliza en forma explícita una estructura de repetición
- Recursión: llamadas repetidas a funciones

Terminación

- Iteración: cuando falla la condición del ciclo
- Recursión: cuando se alcanza el caso base
- Ambas pueden tener ciclos infinitos
 - Iteración: Mayor esfuerzo y cantidad de líneas de código que la recursión.
 - Recursión: Costo muy elevado de performance (Las llamadas recursivas requieren tiempo y consumen memoria adicional)

Balance

- Elección entre performance (iteración) y buenas prácticas de ingeniería de software (recursión).

27

Recursión Infinita

Si no se llega a una llamada que no implica recursión (caso base), cada llamada recursiva produce otra llamada recursiva. Una llamada a la función se ejecutará, en teoría, eternamente.

- Esto se llama recursión infinita.
- En la práctica, una función así se ejecutará hasta que la computadora se quede sin recursos, y el programa terminará anormalmente.

28

Repensemos como funciones recursivas

En grupos repensar las funciones que se indican a continuación:

Iterativa	Recursiva
CantidadDigitos	CantidadDigitos
SumarDigitos	SumarDigitos
Fibonacci	Fibonacci
EsPotenciaDeDos	EsPotenciaDeDos
InvertirNumero	InvertirNumero

29

CantidadDigitos - Recursiva

```
int CantidadDigitos(int numero) {  
    if (numero < 10 ) return 1;  
    else return 1 + CantidadDigitos(numero/10);  
}
```

30

SumarDigitos - Recursiva

```
int SumarDigitos(int numero) {  
    if (numero < 10 ) return numero;  
    else return numero%10 + SumarDigitos(numero/10);  
}
```

31

Fibonacci - Recursiva

```
int fibonacci(int numero) {  
    if (numero == 0)  
        return 0;  
    else if (numero == 1)  
        return 1;  
    else  
        return fibonacci(numero - 1) + fibonacci(numero - 2);  
}
```

32

EsPotenciaDeDos - Recursiva

Si un número n se puede dividir sucesivamente entre 2 hasta llegar a 1, y nunca da un número impar (excepto al final), entonces es potencia de 2.

```
bool esPotenciaDeDos(int n) {
    if (n == 1) return true;           //  $2^0 = 1$ 
    else if (n < 1 || n % 2 != 0)
        return false; // si es menor que 1 o impar (excepto el 1), no es potencia de 2
    else
        return esPotenciaDeDos(n / 2);
}
```

33

Algoritmos y Estructuras de Datos – AEDD – Agenda 18/05

- ✓ Funciones: Ventajas
- ✓ Diseño Modular
- ✓ Tipos de Subprogramas.
- ✓ Funciones.
- ✓ Procedimientos.
- ✓ Parametros: De entrada y De salida y de Entrada/Salida
- ✓ Funciones recursivas

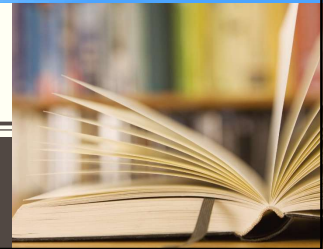
¡¡OBJETIVOS DEL DIA ALCANZADOS!!

34

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¿DUDAS?



35

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¡Buena semana!



36